

PDFForge

Antigravity Phase-by-Phase Implementation Guide

Use this document as your implementation roadmap. Complete one phase, test it, fix it, then move to the next.

Phase	Main focus
1	Foundation: Setup, Auth, Database & Upload
2	Core PDF Operations
3	PDF Editor, Forms & Fill/Sign
4	Compression, OCR, Deskew, Grayscale & Flatten
5	Conversion Suite
6	Security, Watermark & Unlock
7	Advanced PDF Tools
8	Dashboard, History, Files & Settings
9	Admin Panel, RBAC & Analytics
10	Testing, Docker, Security & Deployment

Antigravity rule: Copy only the prompt for the current phase. Do not ask it to build all 10 phases at once. After each phase, run, test and fix the project.

PHASE 1: Foundation: Setup, Auth, Database & Upload

Goal: Build the base full-stack application that all later PDF tools will use.

What to implement

- ☐ React + TypeScript + Vite + Tailwind + shadcn/ui
- ☐ Java 21 + Spring Boot + Maven + MySQL
- ☐ JWT authentication + BCrypt + protected routes
- ☐ Users, files and processing_jobs database structures
- ☐ Reusable drag/drop upload component
- ☐ PDF.js preview with thumbnails, zoom and page navigation
- ☐ Shared ToolLayout, upload, options, progress and result components
- ☐ Tool registry and routes for all planned tools
- ☐ README and .env.example

Copy this prompt into Antigravity — Phase 1

Implement PHASE 1 of PDFForge only.
Set up React + TypeScript + Vite + Tailwind + shadcn/ui and Java 21 + Spring Boot + Maven + MySQL.
Create clean frontend/backend architecture.
Implement registration, login, logout, JWT, BCrypt and protected routes. Create users, files and processing_jobs entities/schema.
Create secure temporary StorageService and reusable drag-and-drop upload. Accept PDFs first. Create PDF.js preview with thumbnails, zoom and page navigation.
Create reusable PDF tool workspace components and routes for every planned tool. Do not implement advanced processing yet.
Run frontend/backend, connect MySQL, test auth, upload and preview, and fix all errors. No fake PDF results.

Testing checklist — do not continue until these pass

- ☐ Frontend runs
- ☐ Backend runs
- ☐ MySQL connects
- ☐ Register/login/logout work
- ☐ Protected API works
- ☐ PDF upload works
- ☐ PDF preview works
- ☐ All tool routes load

PHASE 2: Core PDF Operations

Goal: Implement the most useful page/file operations first.

What to implement

- ☐ Merge multiple PDFs/images
- ☐ Extract selected pages
- ☐ Split by page ranges
- ☐ Delete pages
- ☐ Organize/reorder pages
- ☐ Rotate pages
- ☐ Crop pages
- ☐ Resize pages: A3/A4/A5/Letter/Legal/custom
- ☐ Split common formats in half
- ☐ Secure result downloads

Copy this prompt into Antigravity — Phase 2

Implement PHASE 2 of PDFForge.

Use Apache PDFBox to implement Merge, Extract Pages, Split By Pages, Delete Pages, Organize, Rotate, Crop, Resize and Split In Half.

Build reusable page thumbnails, selection and drag-reordering. Every tool must have upload, preview, options, processing, result and secure download.

Validate page ranges. Store results temporarily and return secure result IDs, never filesystem paths. Enforce authenticated ownership.

Add unit/integration tests and run end-to-end tests before finishing.

Testing checklist — do not continue until these pass

- ☐ Every output opens correctly
- ☐ Page selection/ranges work
- ☐ Reordering works
- ☐ Rotation is permanent
- ☐ Crop/resize work
- ☐ Invalid ranges show errors
- ☐ Cross-user downloads are blocked

PHASE 3: PDF Editor, Forms & Fill/Sign

Goal: Build the interactive editing workspace.

What to implement

- ☐ Three-column editor: thumbnails, canvas, properties
- ☐ Add text/images/shapes/drawing
- ☐ Highlight/underline/strikeout where supported
- ☐ Fillable text, checkbox, radio, dropdown and date fields
- ☐ Draw/upload and place signatures
- ☐ Create Forms workflow
- ☐ Remove supported annotations
- ☐ Preserve original uploads

Copy this prompt into Antigravity — Phase 3

Implement PHASE 3 of PDFForge.

Build PDF Editor, Fill & Sign, Create Forms and Remove Annotations.

Use PDF.js for viewing and an interaction layer for element placement; use PDFBox for final PDF generation where appropriate.

Editor: text, image, rectangle/circle/line, drawing, highlight, underline, strikeout, links where feasible, rotate/delete pages, font/size/alignment/opacity properties.

Fill & Sign: text, checkbox, date, drawn signature and uploaded signature image; move/resize.

Create Forms: text, checkbox, radio, dropdown, date and signature fields with name/required/default settings.

Do not claim perfect editing of arbitrary existing PDF text when unsupported; use reliable overlay/whiteout approaches.

Test every workflow and ensure output PDFs open.

Testing checklist — do not continue until these pass

- ☐ Text saves
- ☐ Images/shapes appear
- ☐ Signature works
- ☐ Forms are fillable
- ☐ Annotations are removed
- ☐ Original PDF remains unchanged

PHASE 4: Compression, OCR, Deskew, Grayscale & Flatten

Goal: Add scanning, OCR and optimization features.

What to implement

- ☐ Compress: low/medium/high
- ☐ Show real size reduction
- ☐ OCR to searchable PDF
- ☐ OCR language selection
- ☐ Deskew scanned pages
- ☐ Grayscale conversion
- ☐ Flatten supported forms/annotations
- ☐ Async processing jobs for heavy work

Copy this prompt into Antigravity — Phase 4

Implement PHASE 4 of PDFForge.

Implement Compress, OCR, Deskew, Grayscale, Flatten and text extraction.

Use real compression. Integrate Tesseract or another legitimate OCR engine; never fake OCR. Support language selection.

Deskew should detect page tilt and offer a manual angle fallback. Grayscale must create valid PDFs. Flatten supported form fields/annotations and communicate limitations.

Implement ProcessingJob states QUEUED, PROCESSING, COMPLETED and FAILED for heavy operations. Do not use hardcoded fake progress.

Add tests and performance safeguards.

Testing checklist — do not continue until these pass

- ☐ Compression works
- ☐ OCR text is searchable
- ☐ Deskew corrects tilt
- ☐ Grayscale output opens
- ☐ Flatten works where supported
- ☐ Heavy jobs do not block requests

PHASE 5: Conversion Suite

Goal: Implement real PDF/document/image conversions.

What to implement

- ☐ PDF → Word
- ☐ PDF → Excel/CSV where appropriate
- ☐ PDF → JPG/PNG/TIFF
- ☐ PDF → PowerPoint
- ☐ PDF → Text
- ☐ Word → PDF
- ☐ JPG/PNG → PDF
- ☐ HTML → PDF
- ☐ Page ranges, quality/DPI and ZIP where applicable

Copy this prompt into Antigravity — Phase 5

Implement PHASE 5 of PDFForge.

Implement PDF→DOCX, PDF→XLSX/CSV, PDF→JPG/PNG/TIFF, PDF→PPTX, PDF→TXT, DOCX→PDF, JPG/PNG→PDF and HTML→PDF.

Use PDFBox, Apache POI, LibreOffice headless and secure conversion libraries where appropriate. Validate inputs/outputs.

Support page ranges, image quality/DPI and ZIP downloads for multiple images. If HTML URL conversion is included, make it safe.

Do not claim perfect layout preservation for complex conversions. Use async jobs for expensive conversions. Add representative tests for text PDFs, tables, DOCX and images.

Testing checklist — do not continue until these pass

- ☐ Target files are produced
- ☐ Outputs open correctly
- ☐ ZIP works
- ☐ Invalid inputs rejected
- ☐ Limitations are honest

PHASE 6: Security, Watermark & Unlock

Goal: Complete security-sensitive document tools responsibly.

What to implement

- ☐ Strong PDF password protection
- ☐ Supported print/copy/edit permissions
- ☐ Unlock using required password when needed
- ☐ Text/image watermark
- ☐ Watermark opacity/size/rotation/position
- ☐ All or selected pages
- ☐ Security audit events without secrets

Copy this prompt into Antigravity — Phase 6

Implement PHASE 6 of PDFForge.
Implement Protect, Unlock, Watermark and finalize Flatten.
Protect with strong PDF encryption and supported permissions.
Unlock must never brute-force passwords or crack documents. Require the correct password when necessary and only remove restrictions legitimately.
Watermark: text/image, font/size/opacity/rotation, common positions, all or selected pages.
Never log passwords/tokens/secrets. Add confirmation for security-sensitive actions and tests for all outputs.

Testing checklist — do not continue until these pass

- ☐ Protected PDF asks for password
- ☐ Wrong unlock password fails safely
- ☐ Watermark placement works
- ☐ Secrets never enter logs
- ☐ Flatten works where supported

PHASE 7: Advanced PDF Tools

Goal: Implement the remaining specialized tools.

What to implement

- ☐ Alternate & Mix
- ☐ Split By Bookmarks
- ☐ Split By Size
- ☐ Split By Text
- ☐ Bates Numbering
- ☐ Create Bookmarks
- ☐ Edit Metadata
- ☐ Extract Images
- ☐ Flip
- ☐ Header & Footer
- ☐ N-Up
- ☐ Page Numbers
- ☐ Rename
- ☐ Repair

Copy this prompt into Antigravity — Phase 7

Implement PHASE 7 of PDFForge.

Implement Alternate & Mix, Split By Bookmarks, Split By Size, Split By Text, Bates Numbering, Create Bookmarks, Edit Metadata, Extract Images, Flip, Header & Footer, N-Up, Page Numbers, Rename and Repair.

Bates: prefix, starting number, digit width, position and selected/all pages.

Bookmarks: nested bookmarks, destination page, reorder/delete.

Metadata: title, author, subject, keywords, creator, producer.

N-Up: 2/4/6/9/16-up with orientation/margins.

Split By Size must calculate actual output groups near the requested maximum.

Rename should safely derive a filename from detected text with manual override.

Repair may attempt legitimate PDFBox recovery but must never falsely claim success.

Create dedicated pages, reuse shared components and add tests.

Testing checklist — do not continue until these pass

- ☐ All tools produce valid output
- ☐ Bates increments correctly
- ☐ Bookmarks open correctly
- ☐ Metadata persists
- ☐ N-Up layout works
- ☐ Repair reports failure honestly when needed

PHASE 8: Dashboard, History, Files & Settings

Goal: Turn the toolkit into a complete user-facing application.

What to implement

- ☐ Dashboard with recent files/operations
- ☐ Processing history
- ☐ Favorite tools
- ☐ User file manager
- ☐ Secure deletion
- ☐ Automatic temporary-file cleanup
- ☐ Storage usage
- ☐ Profile and password change
- ☐ Light/dark/system theme

Copy this prompt into Antigravity — Phase 8

Implement PHASE 8 of PDFForge.
Build Dashboard, Files, History, Favorites, Profile and Settings.
Dashboard: recent operations/files, favorite tools and storage usage.
History: tool, original filename, result filename, status and date/time.
Files: only authenticated user's files/results; secure download/delete; never expose storage paths.
Add scheduled cleanup of expired temporary files.
Settings: profile, password change, light/dark/system theme and optional language.
Use pagination and efficient queries. Enforce ownership on every endpoint.

Testing checklist — do not continue until these pass

- ☐ Dashboard shows only user data
- ☐ History records jobs
- ☐ Files can be deleted
- ☐ Expired files are cleaned
- ☐ Unauthorized access is blocked
- ☐ Settings persist

PHASE 9: Admin Panel, RBAC & Analytics

Goal: Create a secure administrative monitoring area.

What to implement

- ☐ ADMIN role
- ☐ Admin-only APIs
- ☐ User management
- ☐ Processing statistics
- ☐ Failed-job monitoring
- ☐ Storage statistics
- ☐ Most-used tools
- ☐ Audit logs
- ☐ System health

Copy this prompt into Antigravity — Phase 9

Implement PHASE 9 of PDFForge.

Create ADMIN role-based access control and an admin dashboard.

Show real metrics: total users, active users, total/completed/failed jobs, storage usage and most-used tools.

Admin can search users, view details, disable/enable accounts, inspect processing jobs, view safe errors, audit logs and system health.

Protect every admin API server-side; hiding React buttons is not security. Add pagination, authorization tests and audit logging. Never expose passwords, JWT secrets or unnecessary private file contents.

Testing checklist — do not continue until these pass

- ☐ Normal users cannot call admin APIs
- ☐ Metrics are real
- ☐ Sensitive actions are audited
- ☐ Secrets are not logged

PHASE 10: Testing, Docker, Security & Deployment

Goal: Make the application reliable and deployable.

What to implement

- ☐ Backend unit/integration tests
- ☐ Frontend tests
- ☐ Authentication/authorization tests
- ☐ File ownership tests
- ☐ PDF/conversion tests
- ☐ Frontend/backend Dockerfiles
- ☐ docker-compose.yml with MySQL
- ☐ Production environment configuration
- ☐ Security hardening
- ☐ Health checks
- ☐ Deployment/backup documentation

Copy this prompt into Antigravity — Phase 10

Implement PHASE 10, final hardening of PDFForge.
Complete backend and frontend tests plus critical end-to-end tests.
Harden file validation, path traversal protection, command execution, CORS, secure headers, secret handling, ownership checks, rate limiting strategy and temporary-file cleanup. Secure LibreOffice/Tesseract invocation.
Create frontend/backend Dockerfiles and docker-compose.yml with MySQL volume, environment variables and health checks.
Complete README, .env.example, API docs, database setup, local development, Docker deployment, production notes and backup/restore guidance.
Run from a clean environment and fix reproducible compilation, migration and runtime errors. Remove all fake functionality.

Testing checklist — do not continue until these pass

- ☐ Docker Compose starts
- ☐ Frontend production build succeeds
- ☐ Backend production build succeeds
- ☐ Database initializes
- ☐ Auth/ownership tests pass
- ☐ Major tools pass E2E
- ☐ No secrets committed
- ☐ README works for a new developer

FINAL MASTER CHECKLIST

- ☐ Homepage and All Tools page are polished and responsive.
- ☐ All planned tools have working routes.
- ☐ Core PDF tools perform real operations.
- ☐ Conversion tools produce valid target files.
- ☐ PDF.js preview works.
- ☐ Authentication and authorization are enforced server-side.
- ☐ Users can access only their own files, results and history.
- ☐ Temporary files expire automatically.
- ☐ Heavy jobs use asynchronous processing where appropriate.
- ☐ Errors are user-friendly and do not expose stack traces.
- ☐ Light/dark/system theme works.
- ☐ Mobile layouts work at common phone/tablet widths.
- ☐ Keyboard navigation and basic accessibility are implemented.
- ☐ Admin routes use RBAC.
- ☐ Docker Compose starts the stack.
- ☐ README and .env.example are complete.
- ☐ No fake functionality remains.
- ☐ Known limitations are documented honestly.

If Antigravity starts changing completed work: "Work only on the current phase. Do not modify completed phases unless required to fix a regression. Preserve existing API contracts and shared components."